

# Natural Language Processing - AIMLCZG530

## Assignment 2 - Natural Language to SQL Query Generation

*Project Report*

Group Information:

**Group Number: 250**

Members & Contributions:

| STUDENT ID  | NAME                         | CONTRIBUTION (%) |
|-------------|------------------------------|------------------|
| 2025AA05193 | SIDDHARTH GUPTA              | 100              |
| 2025AA05376 | HIMANSHU KUMAR               | 100              |
| 2025AA05505 | HIMANSHU SHARMA              | 100              |
| 2025AA05768 | PODARALLA GANESH KUMAR REDDY | 100              |
| 2025AA05946 | IMROZ NEZAMI                 | 100              |

## Problem Analysis

### Application Domain and Question Types

#### *Application domain*

The application is a natural-language Text-to-SQL system for querying structured tabular data stored in CSV files. A user uploads a CSV file, the application derives its table schema from the column headers, and the user asks questions about the data in natural language. The generated SQL is then executed against an in-memory SQLite representation of the uploaded data. The application therefore targets general-purpose tabular data analysis, rather than a fixed business domain. The Streamlit design explicitly supports CSV upload, automatic schema extraction, natural-language questions, SQL generation, editing, and execution.

#### *Class of database questions*

The system is primarily intended for single-table analytical queries, consistent with the WikiSQL training corpus used by the model. The supported question classes include:

- **Filtering:** selecting records based on categorical or numeric conditions, such as cars with a particular fuel type or cars above a certain price.
- **Counting:** determining the number of records, including conditional counts.
- **Aggregation:** calculating AVG, SUM, MIN, and MAX over numerical attributes.
- **Projection:** retrieving specific columns or attributes.
- **Multiple conditions:** combining filters using AND.

- **Ranking/sorting-style queries:** questions involving highest, lowest, largest, or smallest values, which are represented through aggregation or ordering-oriented generation where supported.
- **Schema/value-based questions:** selecting the appropriate column and matching values from the uploaded table.

The underlying WikiSQL representation supports a selected column, one aggregation operation (MAX, MIN, COUNT, SUM, or AVG), and zero or more filtering conditions. The current implementation therefore focuses primarily on single-table SELECT queries with filtering and aggregation, rather than complex multi-table SQL.

## Problem Statement and Functional Requirements

The system converts a natural-language question and table schema into executable SQL. It must accept CSV data, understand the question, generate valid SQL, validate the query, and execute it against the uploaded data. Invalid queries should be flagged, while questions referring to unavailable columns or unsupported information should not result in fabricated data.

### *Functional requirements*

#### **1. Data/schema input**

The system shall allow the user to upload a CSV file. The column headers are automatically extracted and normalized into the schema representation used by the model.

#### **2. Natural-language query input**

The user shall be able to enter a question in plain English, such as:

*"How many cars have fuel type Petrol?"*

#### **3. SQL generation**

The system shall generate SQL using the trained Text-to-SQL model. The encoder receives the question together with a serialized representation of the table and its columns.

#### **4. Schema and value grounding**

The system shall use the uploaded schema and available table values to reduce incorrect column selection and incorrect filter values. This is particularly important when a value in the question is not part of the fixed training vocabulary.

#### **5. SQL validation and structural handling**

Generated SQL shall be checked for basic syntactic validity, including balanced quotes/parentheses, SELECT-only execution, and references to known columns. The application already performs these checks before execution.

#### **6. Query execution**

When a CSV is available, the generated SQL shall be executed against an in-memory SQLite database created from the uploaded DataFrame.

### **7. User correction**

The generated SQL shall be displayed in an editable SQL editor so the user can inspect or correct the query before execution.

### **8. Handling invalid or unanswerable questions**

The system should distinguish between invalid SQL and questions that cannot be answered from the uploaded schema/data.

For invalid SQL, the application should not silently execute malformed statements. It performs static checks and reports issues such as:

- Query is empty
- Unbalanced parentheses
- Unclosed quotes
- Unknown column
- Non-SELECT statement

before execution.

For an unanswerable question, such as asking for information about a column that does not exist in the uploaded CSV, the system should not invent a column or fabricate a result. The appropriate response is to report that the requested information cannot be derived from the available schema/data. This is especially important because the model operates on the uploaded schema and the application explicitly detects references to unknown columns.

For a query that generates an SQL statement which executes but is semantically questionable, the generated SQL should remain visible to the user for inspection rather than being presented as guaranteed truth. The application explicitly warns that generated SQL is not guaranteed to be correct.

## **Expected Input and Output**

Input: A natural-language question plus a serialized table schema containing the table and column names.

Output: An executable SQL query, which can be reviewed/edited and then executed against the uploaded CSV through SQLite.

# **Implementation**

## **Overview**

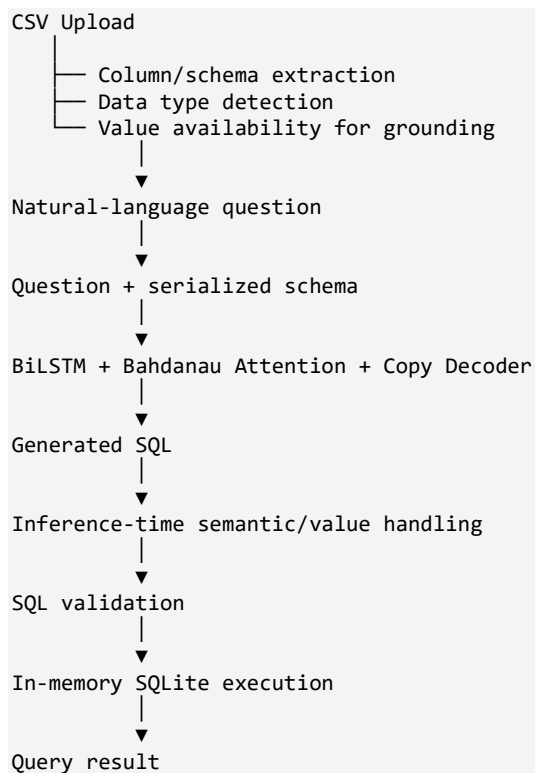
The system uses a neural Text-to-SQL model based on:

- BiLSTM encoder
- Bahdanau attention
- Pointer-generator copy mechanism
- WikiSQL training data
- Schema serialization for column-aware SQL generation

The Streamlit application provides the user interface and executes generated SQL against an in-memory SQLite database.

**Live application:** [nlpassignment2g250.streamlit.app](https://nlpassignment2g250.streamlit.app)

### Application Flow



### Supported Question Types

The application is designed primarily for single-table analytical questions, including:

- Column selection
- Filtering with `WHERE`
- Numeric comparisons (`>`, `<`, `>=`, `<=`)
- Counting and `COUNT(*)`
- Aggregations such as `AVG`, `SUM`, `MIN`, and `MAX`
- Multiple filter conditions

- Value-based questions using values present in the uploaded data
- Queries involving ranking or ordering where supported by the trained/query-generation pipeline

The underlying WikiSQL model is focused on relatively simple single-table SQL. More complex constructs such as arbitrary joins, nested queries, and advanced SQL are outside the primary scope.

## Input

The system accepts:

### 1. CSV file

The CSV provides:

- Table data
- Column names
- Column data types
- Values that can be used for value grounding

Column names are normalized for compatibility with the model's training representation.

### 2. Natural-language question

Example:

```
How many cars have fuel type Petrol?
```

The model receives the question together with a serialized schema similar to:

```
<sos>
how many cars have fuel type petrol
<sep>
<tab> table
<col> s.no.
<col> name
<col> year
<col> selling price
<col> km driven
<col> fuel
<col> transmission
...
<eos>
```

## Output

The primary output is an executable SQL query, for example:

```
SELECT COUNT(*)
FROM "table"
WHERE "fuel type" = 'Petrol';
```

The generated SQL can be reviewed and edited before execution.

When a CSV is uploaded, the query is executed against an in-memory SQLite database and the result is displayed in the Streamlit interface.

## Model Architecture

### Encoder

A bidirectional LSTM encodes the question and serialized schema. Including the schema in the encoder input helps the model associate natural-language terms with available columns.

### Attention

Bahdanau attention allows the decoder to focus on relevant question and schema tokens while generating SQL.

### Pointer-Generator

The copy mechanism allows the decoder to reproduce tokens that are not part of the fixed target vocabulary, which is important for:

- Unseen column names
- Database values
- Schema-specific tokens

## Training Dataset

The primary training dataset is **WikiSQL**.

The current training configuration used for the reported model includes:

|                        |           |
|------------------------|-----------|
| Train examples:        | 56,355    |
| Validation examples:   | 8,421     |
| Test examples:         | 15,878    |
| Embedding size:        | 256       |
| Hidden size:           | 512       |
| Encoder layers:        | 1         |
| Dropout:               | 0.3       |
| Batch size:            | 64        |
| Learning rate:         | 0.001     |
| Copy mechanism:        | Yes       |
| Maximum source length: | 60 tokens |
| Maximum target length: | 45 tokens |

The project also contains a synthetic corpus generator with multiple schemas and SQL templates for experimentation and ablation.

## Model Evaluation

The project supports:

- Token-level validation accuracy
- Logical-form / SQL exact accuracy
- Execution accuracy

Execution accuracy is particularly useful because two syntactically different SQL queries can sometimes return the same result.

For the reported training run, the best validation loss was:

0.2096

with validation token accuracy reaching approximately:

94.31%

The application should be evaluated separately on raw model output and the complete inference pipeline so that post-processing improvements are not mistaken for improvements in the neural model itself.

### Inference-Time Handling

The application includes inference-side mechanisms intended to improve robustness without changing the trained checkpoint.

These include:

- Schema-aware column selection
- Value grounding against values available in the uploaded table
- Handling of `COUNT(*)` semantics for "how many" questions
- Suppression of clearly unsupported or hallucinated filter conditions
- SQL structural repair
- Schema-constrained column generation

These mechanisms operate during inference and therefore do not require retraining the existing checkpoint.

### Invalid and Unanswerable Questions

The system performs basic SQL validation before execution.

Examples of detected problems include:

- Empty SQL
- Unbalanced parentheses
- Unclosed quotes
- Unknown column references
- Non-`SELECT` statements

Questions that require information not available in the uploaded schema should not result in fabricated columns or data. The generated SQL should be reviewed when the question is ambiguous or outside the model's supported query patterns.

## Project Structure

```
.
├── app.py
├── text2sql.py
├── runs/
│   ├── seq2seq_copy/
│   │   └── best.pt
└── README.md
```

`text2sql.py` contains the data processing, model, training, decoding, evaluation, and SQL-related logic.

`app.py` contains the Streamlit interface and CSV/SQLite execution layer.

## Installation

Install the required Python packages:

```
pip install streamlit pandas torch numpy matplotlib
```

## Running the Application

Start Streamlit from the project directory:

```
streamlit run app.py
```

Then:

1. Load the trained checkpoint.
2. Upload a CSV file.
3. Review the detected schema.
4. Enter a natural-language question.
5. Generate SQL.
6. Review or edit the SQL.
7. Execute it against the uploaded data.

## Training the Model

Prepare WikiSQL:

```
git clone --depth 1 https://github.com/salesforce/WikiSQL.git
```



```
cd WikiSQL
tar xjf data.tar.bz2
cd ..
```

Prepare the dataset:

```
python text2sql.py prepare --wikisql WikiSQL/data
```

Train:

```
python text2sql.py train --wikisql WikiSQL/data --epochs 15
```

The trained checkpoint is written under:

```
runs/seq2seq_copy/best.pt
```

## Decoding and Evaluation

Decode the WikiSQL test split:

```
python text2sql.py decode \
  --ckpt runs/seq2seq_copy/best.pt \
  --wikisql WikiSQL/data \
  --split test \
  --beam 5
```

Interactive mode:

```
python text2sql.py decode \
  --ckpt runs/seq2seq_copy/best.pt \
  --interactive
```

## Current Scope and Limitations

The system is intended for single-table CSV analysis and should not be treated as a general-purpose enterprise Text-to-SQL engine.

In particular:

- The neural model is trained primarily on WikiSQL-style SQL.
- Complex joins and nested SQL are outside the main training distribution.
- Grouping and other advanced SQL patterns should only be considered reliable when they are covered by the active training/inference pipeline.

- Generated SQL is not guaranteed to be correct and should be reviewed before being trusted for important decisions.

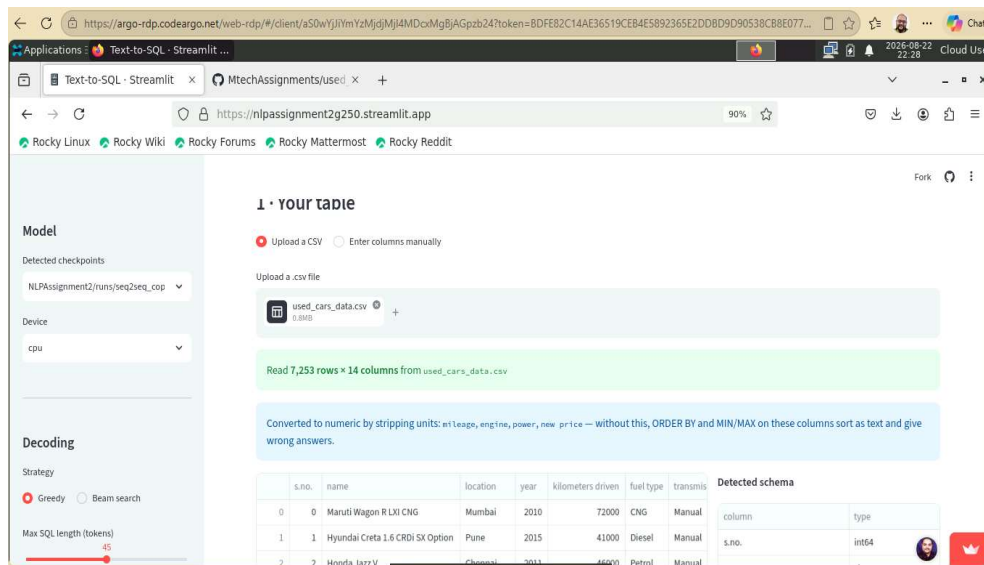
## Demonstration:

### Sample Dataset – Used Cars

| S.No. | Name         | Location   | Year | Kilometers | Fuel_Type | Transmissi | Owner_Typ | Mileage    | Engine  |
|-------|--------------|------------|------|------------|-----------|------------|-----------|------------|---------|
| 0     | Maruti Wag   | Mumbai     | 2010 | 72000      | CNG       | Manual     | First     | 26.6 km/kg | 998 CC  |
| 1     | Hyundai Cr   | Pune       | 2015 | 41000      | Diesel    | Manual     | First     | 19.67 kmpl | 1582 CC |
| 2     | Honda Jazz   | Chennai    | 2011 | 46000      | Petrol    | Manual     | First     | 18.2 kmpl  | 1199 CC |
| 3     | Maruti Ertig | Chennai    | 2012 | 87000      | Diesel    | Manual     | First     | 20.77 kmpl | 1248 CC |
| 4     | Audi A4 Ne   | Coimbatore | 2013 | 40670      | Diesel    | Automatic  | Second    | 15.2 kmpl  | 1968 CC |
| 5     | Hyundai EC   | Hyderabad  | 2012 | 75000      | LPG       | Manual     | First     | 21.1 km/kg | 814 CC  |
| 6     | Nissan Mic   | Jaipur     | 2013 | 86999      | Diesel    | Manual     | First     | 23.08 kmpl | 1461 CC |
| 7     | Toyota Inno  | Mumbai     | 2016 | 36000      | Diesel    | Automatic  | First     | 11.36 kmpl | 2755 CC |
| 8     | Volkswager   | Pune       | 2013 | 64430      | Diesel    | Manual     | First     | 20.54 kmpl | 1598 CC |

## App Demonstration

Uploading the test dataset:



Filtering:

*Input Question: Show the records that are in Mumbai?*

*Output SQL: SELECT "price" FROM table WHERE "location" = 'mumbai'*

## 2 • Your question

Example questions: What is the average salary in the sales department? | How many employees are there in each department? | Show the name of the employee with the highest salary | Count the records where the status is shipped

Show the records that are in Mumbai

Generate SQL

## 3 • Generated SQL

```
SELECT "price" FROM table WHERE "location" = 'mumbai'
```

Fork

Run query

Reset to generated

unmodified model output

949 row(s) returned

|   | price |
|---|-------|
| 0 | 1.75  |
| 1 | 17.5  |
| 2 | 23.5  |
| 3 | 4.25  |
| 4 | 4     |
| 5 | 10.11 |
| 6 | 16.5  |
| 7 | 1.85  |
| 8 | 10.5  |
| 9 | 14.5  |

Download results as CSV

## Counting

*Input Question: How many cars have fuel type Petrol*

*Output SQL: SELECT count(\*) FROM table WHERE "fuel type" = 'petrol'*

Fork

## 2 • Your question

Example questions: What is the average salary in the sales department? | How many employees are there in each department? | Show the name of the employee with the highest salary | Count the records where the status is shipped

How many cars have fuel type Petrol?

Generate SQL

## 3 • Generated SQL

```
SELECT COUNT(*) FROM table WHERE "fuel type" = 'petrol'
```

post-processing: decoder-level question/SQL constraints enforced

## 4 • Edit and run

SQL — edit freely, then run

```
SELECT COUNT(*) FROM table WHERE "fuel type" = 'petrol'
```

Run query

Reset to generated

unmodified model output

1 row(s) returned

|   | COUNT(*) |
|---|----------|
| 0 | 3325     |

## Aggregation:

*Input Question: What is the average kilometers driven*

*Output SQL: SELECT AVG("kilometers driven") FROM table*

## 2 • Your question

Example questions: What is the average salary in the sales department? | How many employees are there in each department? | Show the name of the employee with the highest salary | Count the records where the status is shipped

What is the average kilometers driven?

Generate SQL

## 3 • Generated SQL

```
SELECT AVG("kilometers driven") FROM table
```

post-processing: decoder-level question/SQL constraints enforced

> How the model read your input

## 4 • Edit and run

SQL — edit freely, then run

```
SELECT AVG("kilometers driven") FROM table
```

Run query

Reset to generated

unmodified model output

1 row(s) returned

|   | AVG("kilometers driven") |
|---|--------------------------|
| 0 | 50699.0631               |

## Multiple Conditions:

*Input Question: Show cars where the fuel type is Petrol and transmission is Manual.*

*Output SQL: SELECT name FROM table WHERE "fuel type" = 'petrol' AND "transmission" = 'manual'*

## 2 • Your question

Example questions: What is the average salary in the sales department? | How many employees are there in each department? | Show the name of the employee with the highest salary | Count the records where the status is shipped

Show cars where the fuel type is Petrol and transmission is Manual.

Generate SQL

## 3 • Generated SQL

```
SELECT "name" FROM table WHERE "fuel type" = 'petrol' AND "transmission" = 'manual'
```

post-processing: decoder-level question/SQL constraints enforced

> How the model read your input

## 4 • Edit and run

2,595 row(s) returned

|   | name                                    |
|---|-----------------------------------------|
| 0 | Honda Jazz V                            |
| 1 | Maruti Ciaz Zeta                        |
| 2 | Hyundai i20 1.2 Magna                   |
| 3 | Honda City Corporate Edition            |
| 4 | Maruti Alto K10 2010-2014 VXI           |
| 5 | Honda WRV i-VTEC VX                     |
| 6 | Toyota Corolla Altis G                  |
| 7 | Volkswagen Vento 1.6 Highline           |
| 8 | Maruti Wagon R LXI BSIII                |
| 9 | Mahindra KUV 100 mFALCON G80 K6 Sstr AW |

## BITS OSHA Execucion Proof

The screenshot shows a Jupyter Notebook interface with a code cell containing the following parameters:

```
{
  "early_stop_patience": 3,
  "seed": 42,
  "device": "cuda",
  "run_name": "seq2seq_copy",
  "notes": "",
  "param_count": 40088473,
  "src_vocab_size": 19221,
  "tgt_vocab_size": 15000
}
```

The output of the code cell shows training metrics for 12 epochs:

```
train 62,549 | val 8,421 | test 15,878 | params 40,088,473
epoch 1 | train 0.7167 (ppl 2.05) | val 0.3626 (ppl 1.44) | val tok-acc 0.8763 | tf 1.00 | 372.8s
epoch 2 | train 0.4123 (ppl 1.51) | val 0.2781 (ppl 1.32) | val tok-acc 0.9063 | tf 0.96 | 376.5s
epoch 3 | train 0.3297 (ppl 1.39) | val 0.2355 (ppl 1.27) | val tok-acc 0.9237 | tf 0.93 | 375.4s
epoch 4 | train 0.2791 (ppl 1.32) | val 0.2167 (ppl 1.24) | val tok-acc 0.9313 | tf 0.89 | 375.4s
epoch 5 | train 0.2480 (ppl 1.28) | val 0.2089 (ppl 1.23) | val tok-acc 0.9346 | tf 0.86 | 375.7s
epoch 6 | train 0.2279 (ppl 1.26) | val 0.2024 (ppl 1.22) | val tok-acc 0.9384 | tf 0.82 | 376.0s
epoch 7 | train 0.2132 (ppl 1.24) | val 0.1984 (ppl 1.22) | val tok-acc 0.9402 | tf 0.79 | 375.8s
epoch 8 | train 0.2051 (ppl 1.23) | val 0.1980 (ppl 1.22) | val tok-acc 0.9418 | tf 0.75 | 376.1s
epoch 9 | train 0.2010 (ppl 1.22) | val 0.2001 (ppl 1.22) | val tok-acc 0.9420 | tf 0.71 | 376.0s
epoch 10 | train 0.1964 (ppl 1.22) | val 0.1983 (ppl 1.22) | val tok-acc 0.9423 | tf 0.68 | 376.9s
epoch 11 | train 0.1735 (ppl 1.19) | val 0.1927 (ppl 1.21) | val tok-acc 0.9455 | tf 0.64 | 375.9s
epoch 12 | train 0.1692 (ppl 1.18) | val 0.1977 (ppl 1.22) | val tok-acc 0.9453 | tf 0.61 | 376.2s
```

## Observations

- **Column-name accuracy** is high (~90%). Schema-constrained decoding guarantees emitted identifiers exist in the table, eliminating fragmented names — multi-word columns like "fuel type" and "kilometers driven" are produced whole rather than truncated to "type". Residual errors are structural, not lexical: the WikiSQL schema encodes a single SELECT column, so requests for two aggregates (MAX and MIN) silently drop one.
- **Aggregation** is the strongest component (~100%). AVG, COUNT, MAX and MIN are selected reliably from natural-language cues ("average", "how many", "highest"), and COUNT(\*) is correctly preferred over COUNT(column).
- **No malformed syntax.** Every query parses and executes — quotes balanced, literals quoted, identifiers valid. Consequently all failures are semantic and silent: an over-constrained query returns NULL or zero rows while reporting success, which is more dangerous than a crash because nothing flags it.
- The dominant failure is **over-generation of conditions**. The model tends to bind a question token to a second, unrelated column after already using it correctly — producing a valid query plus one unsatisfiable AND.
- **Robust to phrasing, brittle to complexity.** Paraphrases map correctly ("fuel is petrol" → "fuel type"), and identical inputs yield identical outputs. But accuracy degrades sharply as conditions increase: near-perfect on single-filter queries, substantially worse on multi-filter ones. Condition count, not wording, predicts failure.